


Centro Federal de Educação Tecnológica de Minas Gerais

Engenharia de Computação

Computação Gráfica

# Relatório de Desenvolvimento

Simulação de um Braço Robótico Articulado

João Paulo da Cunha Faria

Maria Eduarda Teixeira Souza

Divinópolis - MG

Agosto de 2026

# 1 Introdução

A modelagem de manipuladores robóticos é um passo fundamental no estudo da robótica, permitindo prever o posicionamento espacial de suas ferramentas. Ao transpor a complexidade da matemática pura para a simplicidade das ferramentas de processamento gráfico, observa-se que as bibliotecas fornecem uma abstração robusta sobre os algoritmos matemáticos.

Para um sistema de braço robótico articulado operando em um plano bidimensional, a posição de cada junta pode ser determinada por meio das equações de cinemática direta, utilizando trigonometria $(\cos(\theta), \sin(\theta))$.

# 2 Modelagem Matemática e Primitivas Gráficas

O projeto computacional exige o cálculo do posicionamento das articulações e sua posterior renderização na tela. Conforme abordado nas aulas da disciplina, há uma divisão clássica entre o Plano Cartesiano e o Plano de Tela (*Screen Space*).

Devido à herança da arquitetura dos monitores CRT (Tubo de Raios Catódicos), a origem de coordenadas $(0, 0)$ de matrizes gráficas em linguagens como Python, C e Java situa-se sempre no canto superior esquerdo, e o eixo $Y$ cresce para baixo. Dessa forma, para que a renderização do robô aponte para "cima"na tela, é estritamente necessário subtrair o resultado do cálculo trigonométrico do eixo $Y$ da base, caso contrário, o robô seria desenhado de ponta-cabeça.

Assim, a posição final $(x_f, y_f)$ de uma junta conectada a um elo de comprimento $L$ sob um ângulo $\theta$ em relação à sua origem $(x_i, y_i)$ é adaptada ao *Screen Space* da seguinte forma:

$$x_f = x_i + L\cos(\theta) \tag{1}$$

$$y_f = y_i - L\sin(\theta) \tag{2}$$

## 2.1 Posicionamento com Cinemática Direta

O braço é composto por uma base, um ombro, um cotovelo e uma ferramenta na extremidade (garra). A partir do ponto fixo da base, o cotovelo é posicionado utilizando o comprimento do braço e o ângulo do ombro. Subsequentemente, a mão é posicionada. Um ponto crucial da modelagem é a dependência do ângulo total acumulado: o ângulo do segundo elo (antebraço) deve ser calculado obrigatoriamente utilizando a soma dos ângulos da junta anterior e da junta atual (`ang_ombro_rad + ang_cotovelo_rad`).

## 2.2 Melhorias Implementadas Além do Escopo Base

O desafio do Braço Robótico proposto exigia o cálculo das coordenadas de interseção e o uso de funções primitivas básicas do OpenCV (`cv2.line`, `cv2.circle`, `cv2.rectangle`) para projetar a peça na tela. No entanto, visando aprimorar a estética e aproximar a simulação gráfica de um modelo mecânico real, este trabalho implementou melhorias adicionais estilizadas:

- **Engrenagens Paramétricas:** Ao invés de representar as juntas apenas com círculos preenchidos simples, foi desenvolvida a função `desenhar_engrenagem`. Esta função utiliza conceitos de divisão angular trigonométrica para iterar e alocar $N$ pequenos "dentes"de engrenagem ao redor da junta principal, conferindo um aspecto de atuador rotativo industrial.
- **Garra Direcional (Atuador de Extremidade):** A extremidade do robô recebeu uma garra composta por um círculo (base) e dois segmentos de reta (dedos). Para que a garra sempre aponte na mesma direção do antebraço, o algoritmo usa `math.atan2(dy, dx)` — que converte o vetor cotovelo-mão diretamente em um ângulo, sem precisar recalcular a soma $\theta_1 + \theta_2$. A partir desse ângulo, os dois dedos são desenhados com abertura de $\pm 0.6$ radianos, formando um "V"que acompanha o braço em qualquer configuração.

# 3 Metodologia

Nesta seção são descritos os materiais, a configuração do ambiente e a estrutura do código desenvolvido para a simulação.

## 3.1 Materiais e Configuração

O projeto foi desenvolvido em *Python*, utilizando as bibliotecas `math`, `numpy`, `opencv-python` (`cv2`) e `matplotlib`, instaladas via:

```
pip install opencv-python matplotlib numpy
```

A tela de renderização é alocada como uma matriz `numpy`, posteriormente manipulada pelo OpenCV:

```
tela = np.zeros((400, 400, 3), dtype=np.uint8)
```

## 3.2 Procedimentos e Scripts Utilizados

O algoritmo principal foi modularizado em funções para separar a lógica de cálculo cinemático da lógica de renderização gráfica. As funções criadas são detalhadas a seguir.

A função `calcular_junta` (Listagem 1) implementa o cerne matemático, utilizando trigonometria básica (`math.cos` e `math.sin`) para definir as novas coordenadas, convertendo os resultados para inteiros, exigência das funções de desenho.

Listing 1: Cálculo cinemático das juntas do braço robótico

```
def calcular_junta(x_origem, y_origem, comprimento, angulo_rad):
    x_final = x_origem + comprimento * math.cos(angulo_rad)
    y_final = y_origem - comprimento * math.sin(angulo_rad)
    return int(round(x_final)), int(round(y_final))
```

A função `desenhar_engrenagem` (Listagem 2) é uma melhoria estética. Ela desenha um círculo base e, iterativamente através da divisão dos 360 graus (ou $2\pi$), calcula e posiciona pequenos círculos (dentes) nas bordas do círculo principal, finalizando com um furo central escuro.

Listing 2: Função estilizada para desenho de engrenagens utilizando aproximações trigonométricas

```
def desenhar_engrenagem(tela, centro_x, centro_y, raio, cor, num_dentes=8):
    """Desenha um formato de engrenagem estilizada"""
    # Desenha o circulo base da engrenagem
    cv2.circle(tela, (centro_x, centro_y), raio, cor, -1)

    # Desenha os dentes da engrenagem usando trigonometria
    for i in range(num_dentes):
        angulo = i * (2 * math.pi / num_dentes)

        # Calcula a ponta de cada dente
        dente_x = int(centro_x + (raio * 1.3) * math.cos(angulo))
        dente_y = int(centro_y + (raio * 1.3) * math.sin(angulo))

        # Desenha os dentes como pequenos circulos na borda
        cv2.circle(tela, (dente_x, dente_y), int(raio * 0.4), cor, -1)

    # Desenha um furo no meio (eixo preto/cinza escuro)
    cv2.circle(tela, (centro_x, centro_y), int(raio * 0.4), (50, 50, 50), -1)
```

A função `desenhar_garra` (Listagem 3) é outra melhoria implementada. Ela calcula a orientação global do antebraço usando `math.atan2` para que a garra aponta sempre para a direção correta. A partir disso, dois dedos são gerados com uma abertura de 0.6 radianos.

Listing 3: Função estilizada para desenho da garra do robô

```python
def desenhar_garra(tela, cotovelo_x, cotovelo_y, mao_x, mao_y):
    """Desenha uma garra com 2 dedos apontando na direcao do antebraco"""
    # Calcula o angulo do antebraco para saber para onde a mao aponta
    angulo_braco = math.atan2(mao_y - cotovelo_y, mao_x - cotovelo_x)

    tamanho_dedo = 25
    abertura = 0.6

    # Calcula as posicoes das pontas dos dedos esquerdo e direito
    ponta_esq_x = int(mao_x + tamanho_dedo * math.cos(angulo_braco - abertura))
    ponta_esq_y = int(mao_y + tamanho_dedo * math.sin(angulo_braco - abertura))

    ponta_dir_x = int(mao_x + tamanho_dedo * math.cos(angulo_braco + abertura))
    ponta_dir_y = int(mao_y + tamanho_dedo * math.sin(angulo_braco + abertura))

    # Desenha a base da mao e os dois dedos (linhas grossas)
    cor_garra = (255, 255, 0)
    cv2.circle(tela, (mao_x, mao_y), 10, cor_garra, -1)
    cv2.line(tela, (mao_x, mao_y), (ponta_esq_x, ponta_esq_y), cor_garra, 5)
    cv2.line(tela, (mao_x, mao_y), (ponta_dir_x, ponta_dir_y), cor_garra, 5)
```

A função `desenhar_braco_estilizado` (Listagem 4) encarrega-se de sobrepor e unificar todas as formas geométricas (linhas, retângulos, engrenagens e garra) na matriz `tela`. Após compor a imagem, a biblioteca Matplotlib é utilizada para salvar a renderização final no arquivo `braco_robotico_estilizado.png`.

Listing 4: Função unificadora para desenhar o braço completo e exportar a imagem

```python
def desenhar_braco_estilizado(base_x, base_y, cotovelo_x, cotovelo_y, mao_x,
    mao_y, tela):
    # Primitiva: Retangulo (a base fixa do robo)
    cv2.rectangle(tela, (base_x - 35, base_y), (base_x + 35, 400), (128, 128,
        128), -1)

    # Primitiva: Linha (os elos rigidos)
    cv2.line(tela, (base_x, base_y), (cotovelo_x, cotovelo_y), (200, 200, 200)
        , 8)
    cv2.line(tela, (cotovelo_x, cotovelo_y), (mao_x, mao_y), (200, 200, 200),
        8)

    # Engrenagens
    desenhar_engrenagem(tela, base_x, base_y, raio=12, cor=(255, 0, 0))
    desenhar_engrenagem(tela, cotovelo_x, cotovelo_y, raio=10, cor=(255, 0, 0)
        )

    # Garra
    desenhar_garra(tela, cotovelo_x, cotovelo_y, mao_x, mao_y)

    # Exibe a imagem
    plt.imshow(tela)
    plt.title("Braco Robotico Articulado (Estilizado)")
    plt.savefig("braco_robotico_estilizado.png")
```

A orquestração principal (Listagem 5) gerencia a coleta dos ângulos escolhidos pelo usuário, aloca a tela virtual e chama o fluxo de renderização.

Listing 5: Rotina principal de execução e coleta de dados

```
def definir_angulos():
    base_x, base_y = 120, 340
    comprimento_braco = float(input("Digite o comprimento do braco (em pixels) padrao 130: "))
    comprimento_antebraco = float(input("Digite o comprimento do antebraco (em pixels) padrao 100: "))
    ang_ombro = float(input("Digite o angulo do ombro (em graus) padrao 65: "))
    ang_cotovelo = float(input("Digite o angulo do cotovelo (em graus) padrao -40: "))

    ang_ombro_rad = math.radians(ang_ombro)
    ang_cotovelo_rad = math.radians(ang_cotovelo)
    return base_x, base_y, comprimento_braco, comprimento_antebraco, ang_ombro_rad, ang_cotovelo_rad

def main():
    print("Iniciando a projecao do braco robotico...")
    base_x, base_y, comprimento_braco, comprimento_antebraco, ang_ombro_rad, ang_cotovelo_rad = definir_angulos()
    tela = np.zeros((400, 400, 3), dtype=np.uint8)

    cotovelo_x, cotovelo_y = calcular_junta(base_x, base_y, comprimento_braco, ang_ombro_rad)
    mao_x, mao_y = calcular_junta(cotovelo_x, cotovelo_y, comprimento_antebraco, ang_ombro_rad + ang_cotovelo_rad)

    print(f"Ombro   --> Coordenadas: ({base_x}, {base_y})")
    print(f"Cotovelo --> Coordenadas: ({cotovelo_x}, {cotovelo_y})")
    print(f"Mao      --> Coordenadas: ({mao_x}, {mao_y})")

    desenhar_braco_estilizado(base_x, base_y, cotovelo_x, cotovelo_y, mao_x, mao_y, tela)

if __name__ == "__main__":
    main()
```

# 4 Resultados e Análises

Durante a execução da rotina `main()`, os parâmetros angulares e dimensionais podem ser definidos dinamicamente pelo usuário. Assumindo a entrada padrão:

- Coordenadas da base: $(120, 340)$
- Comprimento do elo do ombro: 130 px
- Comprimento do elo do antebraço: 100 px
- Ângulo do ombro: $65°$
- Ângulo relativo do cotovelo: $-40°$

O sistema obteve êxito no cálculo progressivo de cada junta com base nas restrições trigonométricas do desafio. A matriz de transformação implícita propagou adequadamente as coordenadas cartesianas até a posição do efetuador final (garra).

As coordenadas projetadas no console, confirmam o alinhamento exato no plano computacional:

```
Ombro   --> Coordenadas: (120, 340)
Cotovelo --> Coordenadas: (175, 222)
Mão      --> Coordenadas: (266, 180)
```

A renderização visual resultante pode ser conferida na Figura 1, ilustrando o mapeamento de coordenadas matemáticas para os índices da matriz de pixels, respeitando o espaço de tela com a origem no quadrante superior esquerdo.

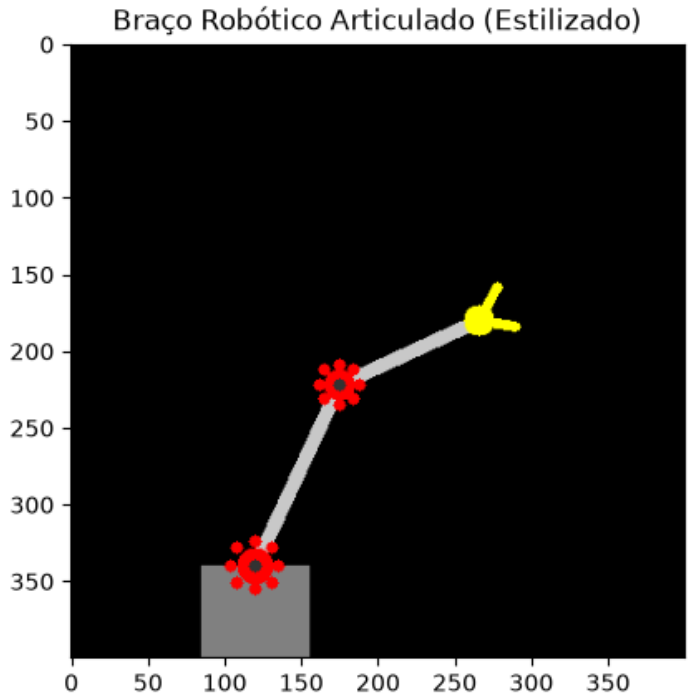



Figura 1: Renderização final do braço robótico estilizado, apresentando a base, os braços e as melhorias aplicadas às juntas (engrenagens) e extremidade (garra).

## 5 Conclusão

O experimento atingiu amplamente os objetivos do Desafio do Braço Robótico. Foi realizada a conversão bem-sucedida de conceitos matemáticos abstratos de cinemática e trigonometria em uma representação gráfica bidimensional estruturada em *Screen Space*. As adições de componentes estilizados demonstraram que o domínio matricial permite o desenho de mecanismos complexos partindo de primitivas básicas.

## Referências

[1] RODRIGO, Raulivan. **Desenhar primitivas: Computação Gráfica**. Notas de Aula (05 - Desenhar primitivas.pdf). Departamento de Computação, Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG), 2026.